JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.file

Class Paths

java.lang.Object
    java.nio.file.Paths

```
public final class Paths
extends Object
```

This class consists exclusively of static methods that return a `Path` by converting a path string or `URI`.

Since:

1.7

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static `Path`	`get(String first, String... more)`	Converts a path string, or a sequence of strings that when joined form a path string, to a `Path`.
static `Path`	`get(URI uri)`	Converts the given URI to a `Path` object.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

get

```
public static Path get(String first,
                       String... more)
```

Converts a path string, or a sequence of strings that when joined form a path string, to a `Path`. If `more` does not specify any elements then the value of the `first` parameter is the path string to convert. If `more` specifies one or more elements then each non-empty string, including `first`, is considered to be a sequence of name elements (see `Path`) and is joined to form a path string. The details as to how the Strings are joined is provider specific but typically they will be joined using the `name-separator` as the separator. For example, if the name separator is `"/"` and `getPath("/foo","bar","gus")` is invoked, then the path string `"/foo/bar/gus"` is converted to a `Path`. A `Path` representing an empty path is returned if `first` is the empty string and `more` does not contain any non-empty strings.

The `Path` is obtained by invoking the `getPath` method of the `default FileSystem`.

Note that while this method is very convenient, using it will imply an assumed reference to the default `FileSystem` and limit the utility of the calling code. Hence it should not be used in library code intended for flexible reuse. A more flexible alternative is to use an existing `Path` instance as an anchor, such as:

```
Path dir = ...
Path path = dir.resolve("file");
```

Parameters:

`first - the path string or initial part of the path string`

`more - additional strings to be joined to form the path string`

Returns:

`the resulting Path`

Throws:

`InvalidPathException - if the path string cannot be converted to a Path`

See Also:

`FileSystem.getPath(java.lang.String, java.lang.String...)`

get

```
public static Path get(URI uri)
```

Converts the given URI to a `Path` object.

This method iterates over the `installed` providers to locate the provider that is identified by the URI `scheme` of the given URI. URI schemes are compared without regard to case. If the provider is found then its `getPath` method is invoked to convert the URI.

In the case of the default provider, identified by the URI scheme "file", the given URI has a non-empty path component, and undefined query and fragment components. Whether the authority component may be present is platform specific. The returned `Path` is associated with the `default` file system.

The default provider provides a similar *round-trip* guarantee to the `File` class. For a given `Path` *p* it is guaranteed that

```
Paths.get(p.toUri()).equals( p.toAbsolutePath())
```

so long as the original `Path`, the `URI`, and the new `Path` are all created in (possibly different invocations of) the same Java virtual machine. Whether other providers make any guarantees is provider specific and therefore unspecified.

Parameters:

`uri - the URI to convert`

Returns:

`the resulting Path`

Throws:

`IllegalArgumentException - if preconditions on the uri parameter do not hold. The format of the URI is provider specific.`

`FileSystemNotFoundException - The file system, identified by the URI, does not exist and cannot be created automatically, or the provider identified by the URI's scheme component is not installed`

C

```
SecurityException - if a security manager is installed and it
denies an unspecified permission to access the file system
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.